INTE 22283 – Mobile Application Development

In-Class Exercise

Mobile Development Platforms & Mobile Application Architecture

Duration	Total Marks
3 hours	100

Instructions

- Case Study 1 draws only on Mobile Development Platforms. Case Study 2 draws only on Mobile Application Architecture. Each case study is self-contained — you do not need Week 4 content to answer Case Study 1, or vice versa.
- Answer by applying the specific facts given in each scenario. Answers that only restate lecture definitions without applying them to the scenario will receive reduced marks.
- Every diagram question requires a labelled sketch (hand-drawn and photographed, or drawn digitally, is acceptable).
- Attempt to study both cases. Work individually unless your instructor states otherwise.

Case Study 1 — FlexiPark: Choosing a Development Approach

Scenario

The University of Kelaniya's IT unit has been asked to build FlexiPark — an app that lets students and staff check real-time parking bay availability, reserve a bay for up to 30 minutes before arrival, pay a small reservation fee, and navigate to the reserved bay via an on-campus map. A separate "Warden" mode lets parking attendants confirm a reservation and mark bays as free again.

Key facts about the project:

- Development team: 3 developers, all comfortable with web technologies (HTML/CSS/JS/React); none has built a native Android or iOS app before.
- Budget: enough to build the app once, not as two separate native versions.
- Deadline: a working pilot on 200 students' phones (a mix of Android and iOS) in 10 weeks.
- Expected lifespan: at least 5 years, with new features (EV charging-bay booking, visitor passes) likely to be added every semester.
- Campus WiFi is unreliable in parts of the car park, so the app should keep working, at least partly, without a live connection.
- Once the pilot succeeds, the university wants to add camera-based number-plate scanning for wardens, which will need fast access to the camera and on-device image processing.

Question 1 — Applying the Decision Framework (20 marks)

(a) Using the Week 3 decision framework, recommend the most suitable development approach for FlexiPark — Native, Hybrid, or Cross-platform. Justify your choice by referring to at least four of: performance, development cost, time-to-market, maintainability, future scalability, and access to new platform features. *[10 marks]*

(b) Explain why each of the other two approaches would be a weaker fit for this specific project, referring to the facts given above (e.g. the team's skills, the 10-week deadline, or the 5-year lifespan). *[10 marks]*

Question 2 — Comparing the Three Approaches for This App (15 marks)

(a) For FlexiPark specifically (not in general), briefly compare Native, Hybrid, and Cross-platform on: performance, development cost, time-to-market, and maintainability. Your comparison should refer to FlexiPark's own numbers and constraints, not generic pros and cons. *[10 marks]*

(b) The university plans to add camera-based number-plate scanning later. Explain what risk this creates for the approach you recommended in Question 1, and how that risk could be managed. *[5 marks]*

Question 3 — Development Tools and Frameworks (15 marks)

(a) If FlexiPark were built using a cross-platform framework, recommend Flutter or React Native for this specific team, and justify your choice by referring to the team's existing skills and to how each framework renders its UI. *[8 marks]*

(b) If FlexiPark were instead built natively, name the IDE and primary programming language needed for the Android version and for the iOS version, and explain one practical consequence this would have for this specific team. *[7 marks]*

Case Study 2 — PulseFit: Structuring the App

Scenario

A health-tech startup has already decided how PulseFit, a companion app for a wrist-worn heart-rate sensor, will be built and is now focused on structuring the code. The app has three screens: Home (live heart rate), Stats (charts of historical data), and Settings. PulseFit must keep logging heart-rate readings in the background even while the app is minimised, must not lose the current reading or the user's place on screen if the phone is rotated, and should store readings locally first, syncing to the cloud when a connection is available.

Key facts about the project:

- The app currently puts all of its logic directly inside the Home screen's Activity class, which is becoming long and hard to test.
- The startup expects to keep adding screens and features over time, and wants the heart-rate logic to be testable without running the full UI.
- Testers have noticed that rotating the phone while on the Home screen sometimes shows a blank or reset heart-rate value for a moment.

Question 1 — MVC vs MVVM (20 marks)

(a) Briefly compare MVC and MVVM, then recommend the better fit for PulseFit. Justify your choice using specific features of this app — for example, the need to keep showing the live reading through a screen rotation, or the wish to test the heart-rate logic independently of the UI. *[8 marks]*

(b) For the Home screen specifically, describe what the Model, the View, and the ViewModel (or Controller, if you chose MVC) would each be responsible for. *[6 marks]*

(c) Draw a simple high-level architecture diagram of the Home screen showing Model, View, and ViewModel/Controller, with arrows showing how data flows between them. *[6 marks]*

Question 2 — Android Components (15 marks)

PulseFit needs to: display the live reading and charts to the user; keep reading and logging heart-rate data even when the app is minimised; and react when the sensor connection drops or reconnects.

(a) For each of the four Android components — Activity, Service, Broadcast Receiver, Content Provider — explain which part of PulseFit (described above) it would be used for, and why it is a good fit for that job. (Not every component needs a major role — briefly note if one has little use here, and why.) *[9 marks]*

(b) Draw a simple diagram showing how your chosen components interact so that heart-rate logging continues correctly while the app is minimised. *[6 marks]*

Question 3 — Lifecycle, State Management, and Data Binding (15 marks)

(a) Explain what happens to the Home screen's Activity when the phone is rotated, and why this explains the blank/reset heart-rate value testers observed. *[5 marks]*

(b) Explain how the architecture you chose in Question 1, together with good state management, would prevent this problem. *[5 marks]*

(c) Explain how data binding would let the Stats screen's chart update itself automatically whenever new heart-rate data arrives, without the developer writing code to manually refresh each part of the screen. *[5 marks]*